Programming Assignment 7: Recursion

Due: Thursday, April 2nd at 11pm CT

Learning Goals

- To conceptualize, implement, test, and debug simple recursive algorithms and recursive backtracking algorithms
- To improve the efficiency of a brute force recursive backtracking algorithm through creative approaches

Assignment Guidelines and Specifications

This is a **pair programming assignment**, which means you can work with a partner on this assignment. You are not required to work with a partner and are welcome to complete the assignment individually if you wish. If you choose to work with a partner, there are some important rules you must follow:

- You should follow these best practices when pair programming.
- Your partner **must be in the same section** as you and have the same TA. **You may not work with a partner who you have already worked with on a previous assignment.**
- You will write **one** program file and include both of your names in the file header. **Only one member of the pair will submit the code to Gradescope.** After submitting the assignment, on the submission page, click "View or edit group" on the upper righthand corner of the screen. Enter the name of the other partner in order to add them to the submission. After doing this, the other partner should get an email confirming that they were added to the submission. **If you do not add your partner to the Gradescope submission, the other member of the pair will not get any credit for completing this assignment.** After adding the other partner, check that both partners are able to see the submission on their Gradescope accounts. This will mean that you have both been correctly linked to the submission.
- If you choose to use late days, both partners must have the required amount of late days. If either partner does not, **both** partners will receive a 0 on this assignment.

For this assignment, there are some additional specifications:

- You may use any classes and methods from the Java standard library that you choose, including the `Character` and `String` classes.
- Some of the public methods already have precondition checking included in the starter code. For those that do not, you must add precondition checking.
- You may (and are encouraged to) create private helper methods as you see fit.

Part 1: Warm-Up

In this part of the assignment, you will implement five stand-alone recursive methods. Some of the methods use recursion to solve problems that could just as easily (if not *more* easily) be solved using simple iteration. However, for educational purposes, you must solve all of the methods recursively. Some methods may require that you use both recursion and iteration, especially algorithms that require some form of recursive backtracking.

Getting Started

Here are the necessary files for this assignment:

- **`Recursive.java`**: This file contains a class with five methods that you need to implement using recursion. You will need to complete this file according to the given specifications and submit this file.
- **`RecursiveTester.java`**: This file contains the provided correctness tests. You will need to write your own tests in this file and submit this file.
- **`DrawingPanel.java`**: This file contains a class responsible for creating simple graphics windows. You will need this class in order to implement the Sierpiński Carpet problem. You should not edit this code and do not need to submit this file.
- **`Stopwatch.java`**: This file contains the Stopwatch class, which you use to measure the time elapsed. You should not edit this code and do not need to submit this file.

1. Doubling Values

Implement the method

```
public int nextIsDouble(int[] data)
```

This method accepts an array of ints as a parameter and returns the number of elements that are followed directly by double that element.

- Example 1: If given the array `{1, 2, 4, 8, 16, 32, 64, 128, 256}`, the method should return the int 8. The first eight elements (1, 2, 4, 8, 16, 32, 64, and 128) are all followed directly by elements that are double their values.
- Example 2: If given the array `{1, 3, 4, 2, 32, 8, 128, -5, 6}`, the method should return the int `0`. There are no elements in the array that are followed directly by an element that is double its value.
- Example 3: If given the array `{1, 0, 0, -5, -10, 32, 64, 128, 2, 9, 18}`, the method should return the int 5. The elements `0`, `-5`, 32, 64 and 9 are all followed directly by elements that are double their values.

Hint: Think about if you need to write a recursive helper method or if the recursion can take place in the method above. To determine this, think about what information you need to keep track of and what information you need to return in each recursive method call.

2. Phone Mnemonics

On traditional telephone keypads, each digit is associated with a set of letters, as shown in the picture to the right.

A mnemonic is a word or phrase formed from a sequence of digits by replacing each digit with one of its associated letters. Before the internet made it easy to look up phone numbers, these were commonly used by businesses to make their phone numbers easier to remember. For example, in some localities, the phone number for a recorded time-of-day message is 637-8687, which corresponds to the word "NERVOUS."

Implement the method

```
public ArrayList<String> listMnemonics(String number)
```

This method accepts a string of digits, `number`, and returns all possible letter combinations (mnemonics) that the number could represent based on the mapping of digits to letters on a traditional telephone keypad. The mnemonics should be stored and returned in an `ArrayList`.

For example, the method call `listMnemonics("623")` should return generate and return an ArrayList containing the following 27 possible letter combinations (note that the order of the `String`s in the list does not matter):

```
MAD MBD MCD NAD NBD NCD OAD OBD OCD
MAE MBE MCE NAE NBE NCE OAE OBE OCE
MAF MBF MCF NAF NBF NCF OAF OBF OCF
```

Here are some hints for this problem:

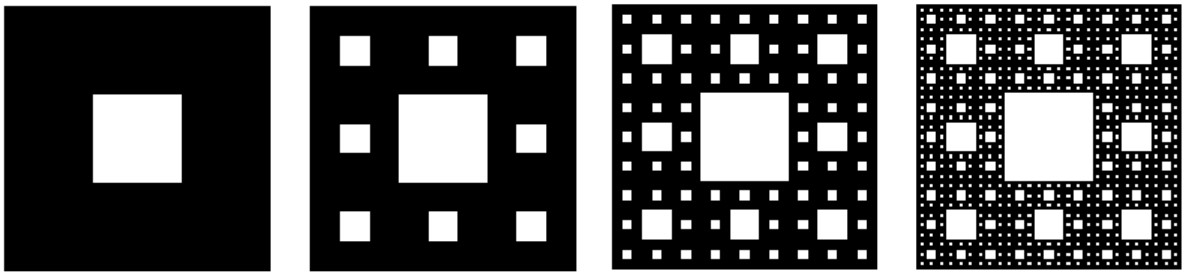
- Think about if you need to write a recursive helper method or if the recursion can take place in the method above. To determine this, think about what information you need to keep track of and what information you need to return in each recursive method call.
- Use the given helper method `digitLetters` to retrieve the letters associated with a given digit.

3. Sierpiński Carpet

A Sierpiński carpet is a geometric pattern generated through the recursive subdivision of a square. If you were to create the pattern by hand, you would do the following:

1. Start with a filled-in square.
2. Divide the square into a 3x3 grid of nine equal-sized smaller squares.
3. "Cut out" the center square (by drawing a white square in that section).
4. Repeat the process for each of the remaining eight squares, recursively removing the center square of each subdivision. You should repeat this process while the size of the squares is greater than some given limit value (that specifies the minimum square size).

Here are a few examples of Sierpinski carpets, with various minimum square sizes:



In the starter code, we have implemented the `drawCarpet` method, which will start the drawing process. You should implement the recursive helper method to complete the carpet:

```
private static void drawSquares(Graphics g, int size, int limit,
                                double x, double y)
```

This method should draw the individual squares of the carpet. There are 5 parameters for this method:

- `Graphics g`: This is the Graphics object you will use to draw the rectangles. The argument for this parameter should always be g. You should not change this argument from one method call to the next.
- `int size`: This is the size of the current squares being drawn.
- `int limit`: This is the minimum allowable square size. You should stop drawing squares once the size of the squares is below this limit.
- `double x`: This is the x-coordinate of the upper lefthand corner of the current square.
- `double y`: This is the y-coordinate of the upper lefthand corner of the current square.

Here are some helpful hints for this problem:

- Recall that the coordinate system for graphics specifies that the top, left corner is coordinate (0, 0). The x-coordinate increases as you move right, and the y-coordinate increases as you go down.
- To draw a rectangle, use the `fillRect` method from the `Graphics` class. When calling this method, remember to cast the x and y-coordinates from `doubles` to `ints`.

4. Flowing Water

100	100	100	100
100	100	100	100
100	100	100	100

In this problem, a map is represented as a 2D array of ints, where each value corresponds to the elevation at that point. Higher numbers indicate higher elevation, and lower numbers indicate lower elevation.

For example, the map to the top right represents a flat plain at an elevation of 100 feet. For another example, a "river" flowing through a gorge would be represented as follows on the bottom right (note that the cells shaded gray make up the river).

100	99	200	200
200	98	200	200
200	97	96	200
200	200	95	200
200	200	94	93
200	150	200	200
200	200	200	200
200	150	100	200
200	200	200	200

On this map, water is able to flow downhill. A drop of water placed at a specific location will move according to the following rules:

1. Water can move to neighboring cells that are up, down, left, or right (no diagonal movement).
2. Water can move to neighboring cells if that cell has a lower elevation (no movement if equal elevation).
3. Water in any cell on the edge of the map can flow off the map.

Implement the method

```
public boolean canFlowOffMap(int[][] map,
                             int row, int col)
```

This method returns true if a drop of water starting at position `(row, col)` can eventually reach the edge of the map and flow off, and returns false if this is not possible.

For example, when using the top-right map, a call to the method using any of the cells on the edge of the map would return `true` because water in any of the edge cells can flow off the map. A call to the method using any of the internal cells will return `false` because water cannot flow across cells with the same elevation (only to a cell with a strictly lower elevation).

For another example, when using the lower-right map, a call to the method for position `(2, 2)` would return `true`. If a drop of water started at the cell at row 2 and column 2, which has an elevation of 96 ft, it would flow to the cell below (95), then the cell below (94), then the cell to the right (93), then flow off the map. A call to the method for position `(3, 1)`, which has an elevation of 200, would also return `true` because the water would flow to the right (95), and then follow the path described in the previous example off the map. A call to the method for position `(6, 1)`, which has an elevation of 200, would return `false`. Water can flow north and south but will eventually dead end and will be unable to reach the edges of the map.

Here are some hints for this problem:

- Think about if you need to write a recursive helper method or if the recursion can take place in the method above. To determine this, think about what information you need to keep track of and what information you need to return in each recursive method call.

5. Creating Fair Teams

Implement the method

```
public static int minDifference(int numTeams, int[] abilities)
```

This method will separate a group of individuals into a specific number of teams, as specified by the first parameter, `numTeams`. Each individual has an assigned *ability score*, which could represent their athletic ability, artistic ability, etc. The ability scores of the individuals are stored in the second parameter, `abilities`. A team's total *ability* is the sum of the ability scores of all of its members.

This method should determine the **minimum possible difference in ability** between the team with the highest ability and the team with the lowest ability, when given a specific number of teams and certain ability scores. There are a few constraints on how teams are generated: 1) every individual (represented by a value in `abilities`) must be assigned to a team, and 2) every team must have at least one member.

For example, assume we want to create three teams using individuals with the following abilities: `[1, 2, 3, 4, 5, 6, 7]`. We need to create three teams such that there is a minimum possible difference in ability between the team with the highest ability and the team with the lowest ability. The program does not need to return or show which individuals are on what team. In this example, the minimum possible difference in ability is created by having two teams with a total ability of 9 and 1 team with an ability of `10`. There are multiple ways of creating teams with these values, one of which is:

Team 1: 1 + 3 + 6 = `10`
Team 2: 2 + 7 = 9
Team 3: 4 + 5 = 9

Therefore, this method should return the int 1, since that is the minimum possible difference.

Here are some hints for this problem:

- Think about if you need to write a recursive helper method or if the recursion can take place in the method above. To determine this, think about what information you need to keep track of and what information you need to return in each recursive method call.
- There is significant work to do when you reach the base case. You must ensure every team has at least one person on it, or else, you have an invalid set of teams. If the setup is invalid, you should return some value (like `Integer.MAX_VALUE`) to indicate the setup is invalid. If you can ensure that the setup is valid, you will need to determine the minimum and maximum team ability scores.

Testing `Recursive.java`

You are required to write and submit your own test cases in `RecrsiveTester.java`. You are expected to write:

- 2 new test cases for `nextIsDouble`
- 2 new test cases for `listMnemonics`
- 2 new test cases for `canFlowOffMap`
- 2 new test cases for `minDifference`

Delete the test cases that we have provided for you in `RecursiveTester.java`.

Part 2: Anagram Solver

Introduction

An anagram is created by rearranging all the letters from a word or phrase to form a new word or phrase. Every letter from the original must be used exactly once, with no additions or omissions. The resulting words must be valid according to a given dictionary. For example, an anagram of "`Gracelynn Ray`" is "`larceny angry`". There are many online anagram solvers you can play around with, like this one.

In this part of the assignment, you will create your very own anagram solver!

- **You must implement the anagram-finding algorithm as outlined in this video explanation.** Do not search the web or use an LLM for algorithms to find anagrams.
- You are not allowed to create a multithreaded application for this assignment. (If you are unaware of what a multithreaded application is, you will learn about this in CS 439.)

Getting Started

Here are the necessary files for this assignment:

- **`LetterInventory.java`**: This class represents a collection of letters from the English alphabet. You will need to write this file entirely on your own, from scratch, according to the given specifications and submit this file.
- **`AnagramSolver.java`**: This class solves and returns a list of anagrams for a given phrase and maximum number of words. You will need to complete this file according to the given specifications and submit this file.
- **`AnagramFinderTester.java`**: This file contains the main method used for testing and the provided correctness tests. You will need to write your own tests in this file and submit this file.
- **`AnagramMain.java`**: This file contains the driver program for the completed anagram solver. Do not alter this file except for printing out time results, if you would like. Do not submit this file.

- **d1.txt**, **d2.txt**, **d3.txt**: These files are example dictionary files used by `AnagramFinderTester`. Do not submit these files.
- **testCaseAnagrams.txt**: This file is used by `AnagramFinderTester` to test your code. Do not submit this file.
- **Stopwatch.java**: This file contains a class for calculating elapsed time when running other code. You may use this to see how long it takes to find anagrams and compare results with other students. Do not submit this file.

`LetterInventory`

Implement a `LetterInventory` class that represents a collection of letters from the English alphabet. A letter inventory object stores the number of times each English letter, `'a'` through `'z'`, occurs in a word or phrase. Note that the letter inventory ignores characters that are not English letters. Also, note that the letter inventory is case insensitive.

For example, the letter inventory of the word `"tall"` is `1 a, 2 l's, and 1 t`.

The letter inventory of the phrase `"Isabelle M. Scott!!"` is `1 a, 1 b, 1 c, 2 e's, 1 i, 2 l's, 1 m, 1 o, 2 s's, and 2 t's`.

You must use an array of `int`s to store the number of times each English letter occurs. The inventory is case insensitive, so the length of the array will equal the length of the alphabet being used (in this case, 26). Use a class constant for the value 26. The class keeps track of the total number of letters in the inventory, so this value can be returned quickly without adding up all the individual counters.

In the `LetterInventory` class, you are required to implement the following methods:

1. A constructor to create a new `LetterInventory` that accepts a String.
    - The precondition requires the String to not be null.
    - This method shall be no worse than `O(N + M)`, where `N` is the number of characters in the String and `M` is the number of letters in the alphabet our `LetterInventory` is using.
    - Recall that you can test if a given character is a letter between `'a'` and `'z'` with the following boolean expression: `'a' <= ch && ch <= 'z'`.

2. A method named `get` that accepts a `char` and returns the frequency of that letter in this `LetterInventory`.
    - The precondition requires the `char` to be an English letter. It may be an upper or lower case letter.
    - The answer returned must be the same given the upper or lower case version of a letter.
    - This method should be `O(1)`.

3. A method named `size` that returns the total number of letters in this `LetterInventory`. This method should be `O(1)`.

4. A method named `isEmpty` that returns `true` if the size of this `LetterInventory` is `0`, `false` otherwise. This method should be `O(1)` with respect to the size of the alphabet.

5. A method named `toString` that returns a String representation of this `LetterInventory`. All letters in the inventory are listed in alphabetical order.

    - For example, if a `LetterInventory` is created from the String `"tall"` toString returns the String `"allt"`.
    - If a `LetterInventory` is created from the String `"Isabelle M. Scott!!"` the method returns the String `"abceeillmosstt"`.

6. A method named `add` with one parameter, another `LetterInventory` object. The method returns a new `LetterInventory` created by adding the frequencies from the calling `LetterInventory` object to the frequencies of the letters in the explicit parameter.

    - The precondition requires that the `LetterInventory` sent as an explicit parameter not be null.
    - The postcondition requires that neither the calling object or the explicit parameter are altered as a result of this method call.
    - This method should be `O(M)`, where `M` is the number of letters in the alphabet our `LetterInventory` is using.

7. A method named `subtract` with one parameter, another `LetterInventory` object. The method returns a new `LetterInventory` object created by subtracting the letter frequencies of the explicit parameter from the calling object's (`this`) letter frequencies.

    - `let1.subtract(let2)` is analogous to `let1 - let2`.
    - If any of the resulting letter counts are less than `0`, the method shall return null.
    - The precondition requires the `LetterInventory` sent as an explicit parameter not be null.
    - The postcondition requires that neither the calling object or the explicit parameter are altered as a result of this method call.
    - This method should be `O(M)`, where `M` is the number of letters in the alphabet our `LetterInventory` is using.

8. A method that **overrides** (**not overloads**) the `equals` method from `Object`. The parameter should be of type `Object`. Two `LetterInventorys` are equal if the frequency for each letter in the alphabet is the same.

You may add other private methods and constructors if you wish.

Note that `LetterInventory` objects are immutable. There are no methods that alter a given `LetterInventory` object after it is created.

When you implement the anagram solver, you will not use all of these methods. However, you are required to complete the class. Your completed `LetterInventory` class must pass all of the tests in `AnagramFinderTester`.

Testing `LetterInventory`

Before moving on, you should thoroughly test the `LetterInventory` class. Any issues that you do not catch now before moving on will only cause bigger problems in the future.

Write your testing code in `AnagramFinderTester.java`. You are expected to write 2 test cases for every public constructor and public method in `LetterInventory`. In other words, you will be writing test cases for the methods we have numbered 1 through 8 above.

Delete the provided tests from `AnagramFinderTester.java`.

`AnagramSolver`

Implement a class to find all possible anagrams for a given phrase. Realize that the legal anagrams for a phrase will vary based on what dictionary is used, the maximum number words allowed in the anagram, and whether words can be repeated in the anagram or not.

For example, the anagrams of `"Isabelle Scott"` using the dictionary in `d3.txt` and a limit of 2 words per anagram are:

```
best, oscillate
bestial, closet
bleat, solstice
blest, societal
closet, stabile
obstacle, stile
solstice, table
```

There are two special cases that you do not have to worry about.

- First, a given word or phrase is considered an anagram of itself if the word is in the dictionary. `"dog"` is an anagram of `"dog"`.
- Second, you may use words more than once in the anagram. For example, one of the anagrams of `"dog dog"` is `"god god"`.

In your solution, each anagram will be represented as a `List<String>`. Your task is to return a `List<List<String>>`, a list of anagrams.

Provide the following for the `AnagramSolver` class:

1. Implement the following public constructor:

```
public AnagramSolver(Set<String> dictionary)
```

   This constructor accepts a `Set<String>` object (a set of strings) as a parameter. The set contains the dictionary that this `AnagramSolver` is to use when forming anagrams.

   This constructor creates and stores letter inventories for each element of the set. Store the original words in the dictionary and their corresponding letter inventories inside the `AnagramSolver` class. You want to be able to access the letter inventory of a given word quickly, so store each word and its associated `LetterInventory` in a `Map`.

   Your class must create the `LetterInventory` for a given word in the dictionary exactly one time. Doing it more than once is an inefficiency you must avoid.

2. Implement the following public method:

```
public List<List<String>> getAnagrams(String s, int maxWords)
```

   This method has two parameters: a `String` and an `int`. The String is the target word or phrase, and your method will form anagrams out of this word or phrase. The String sent as a parameter **does not** have to be in the AnagramSolver's dictionary. It may contain characters which must be ignored (this is handled by the LetterInventory class already).

   The `int` parameter specifies the maximum number of words allowed in the generated anagrams. Only anagrams that do not exceed this limit should be returned. If the integer parameter is `0`, there is no restriction on the number of words in the anagrams. (Logically, however, can you think of a way to limit the maximum number of words in an anagram?)

   The precondition requires the String to not be null and the String contains at least one English letter. The `int` parameter must be greater than or equal to `0`.

   The method returns a `List<List<String>>`. In other words, a `List` of `Lists` of Strings. `List` is a Java interface. The `ArrayList` class implements the `List` interface, so you could return an `ArrayList<ArrayList<String>>`. **Do not alter** the return type from `List<List<String>>`.

   The returned `List<List<String>>` must have the following properties:

   - The words in a particular anagram are sorted in lexicographical order.
   - The list does not contain duplicate anagrams.

- The list of anagrams is sorted with anagrams with the fewest words coming first. For instance, all anagrams with one word come before all anagrams with two words, which come before all anagrams with three words.
- Anagrams with the same number of words are sorted based on the Strings within the anagram. Refer to the sample output for examples.

You can and should add private helper methods as necessary to break the problem up into manageable parts. By way of comparison, the solution `AnagramSolver` class, including a nested class for a `Comparator`, is roughly 135 lines, including comments, blank lines, and lines with a single brace.

Note: You do not need to submit your own test cases for the `AnagramSolver` class.

Preprocessing `getAnagrams`

You should complete some preprocessing each time `getAnagrams` is called. In most cases, many of the words in the dictionary cannot be part of an anagram for the given word or phrase. For example, the word `"queen"` will not be in any anagram of `"Isabelle Scott"` because there is no `'q'` in `"Isabelle Scott."` Likewise, the word `"casa"` cannot be part of an anagram of `"Isabelle Scott"` because there are two `'a'`s in `"casa"` but only `1 'a'` in `"Isabelle Scott."` You should use the `LetterInventory` class to simplify the task of determining if a given word could possibly be part of an anagram. ***Hint*: the subtract method is very useful.**

Generate a smaller list of eligible words from the main dictionary that can be used to form anagrams of the given string. This filtered list must remain separate and must not modify the main dictionary stored in `AnagramSolver`. Once this smaller list is prepared, use it as the set of choices in the recursive backtracking algorithm to find valid anagrams of the given string.

When processing words, do not recreate the `LetterInventory` for each word. Instead, retrieve the precomputed `LetterInventory` from the main dictionary to improve efficiency.

Virtually Shrinking the Dictionary

To improve efficiency, virtually shrink the set of possible words as you go deeper into recursion. Instead of modifying or removing elements from the dictionary, track which words have already been considered and only explore valid choices.

- Avoid creating a new data structure to store a "shrunken" dictionary at each recursive step. Instead, manage how you access existing words dynamically. (What can you do to "track" where you are in the list each recursive call?)
- Ensure each recursive call only considers a subset of words from the original list that are still viable choices. This prevents redundant computations while keeping the dictionary unchanged.

Using Comparators

The returned List<List<String>> has a few sorting properties. Accomplishing this sorting can be a chore. You might try to do it as you go or wait until you have found all the anagrams to sort the list. The second option may be a cleaner solution because it separates the recursive backtracking from the sorting.

You are free to use methods from the Collections class to make this easier, as well as other data structures (the Collections class, not the Collection interface.)

Initially, there is no way to sort a list of anagrams. List and ArrayLists are not Comparable. One approach would be to create a class that implements the Comparator interface. The Comparator interface allows you to define how two objects should be compared if they are not Comparable. You will need a separate class, but do not put it in a separate file. Instead, you can make your class that implements the Comparator interface a nested class, similar to the iterators you have implemented, except this class will be declared static because it does not need to know about the outer class.

```
public class AnagramSolver {
    // code for AnagramSolver

    // example of a nested class
    private static class AnagramComparator implements Comparator<List<String>> {
        public int compare(List<String> a1, List<String> a2) {
            // code for compare
        }
    } // end of AnagramComparator

} // end of AnagramSolver class
```

As a reminder, the compare method for the Comparator interface is similar to the compareTo method for Comparable. If a1 is "less than" a2, return a negative int. If a1 "equals" a2, return 0. If a1 is "greater than" a2, return a positive int.

When you create a class that implements the Comparator interface correctly, you can call the sort method in the Collections class that takes in a list and a comparator to sort the List of Lists of Strings.

Dictionary Files

d1.txt, d2.txt, and d3.txt are dictionary files. d1.txt is a small dictionary file with 56 words. d2.txt is a medium size dictionary file with 3,927 words. d3.txt is a large dictionary file with 19,911 words.

The dictionary files provide valid words for your anagram solver, and AnagramFinderTester relies on a dictionary file to test your code. However, when grading your program, additional

dictionary files will be used, including words of one and two letters. **Do not assume that all words in the dictionary will have a minimum length of three or more.**

Expected Output Files

`AnagramFinderTester` uses `d3.txt` and `testCaseAnagrams.txt` to test your anagram solver. This is the expected output of the tester without the anagrams shown and with the anagrams shown. Your times will vary.

Any of the given tests that complete in less than 0.2 seconds should take less than 1 second on the CS department machines. Any of the given tests that take greater than or equal to 0.2 seconds should take no more than five times the time in the expected output when run on the CS department machines.

Sometimes, solutions are slow because you have missed a base case and made too many calls to your recursive helper method. This is a test output with the number of calls made to the recursive helper method. Note: your number of calls **does not** have to match those shown.

Finally, there is a very clever solution that can significantly improve the time it takes to find anagrams. This is a test output for that approach and the number of calls made to the recursive helper method. Again, your number of calls **does not** have to match those shown.

This is a sample run of anagram solver using `d1.txt`. Other than the timing results, your program **must match this output** given the same input. Many of the anagrams are trimmed for clarity, but when you run `AnagramMain`, it prints out the entire result. The output also shows the timing data for the sample solution and the tests in `AnagramFinderTester.java`.

Here is a run of the anagram solver using `d3.txt` with all of the output.

It is up to you to explore various approaches to try and speed up your algorithm. There is no one right way to do this. It is **up to you** to try different things. Neither the teaching assistants nor the instruction have a "magic answer" that works for all cases. **Do not** come to help hours expecting us to tell you exactly how to speed up your program. We will give high level suggestions at best, not detailed advice. It is **up to you** to create ideas and test them out.

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. For Part 1:
    a. Implement all the required methods in the `Recursive` class?
    b. Ensure your methods in `Recursive` pass the given test cases?
    c. Include your own test cases in the `RecursiveTester` class?
2. For Part 2:
    a. Create your own `LetterInventory` class and implement all the required methods?
    b. Ensure your `LetterInventory` and `AnagramSolver` classes pass the tests in `AnagramFinderTester`?
    c. Include your test cases for `LetterInventory` in `AnagramFinderTester`?
    d. Complete the `AnagramSolver` and implement a recursive backtracking algorithm with the required efficiency improvements?
    e. Ensure your program matches the sample output files when run?
3. Fill in the header comments for all the files?
4. Check the preconditions in public methods and throw exceptions as necessary?
5. Follow the program hygiene guide?
6. Follow the assignment specifications from page 3 of the Assignment handout?

If you have done all these things, then you're ready to submit to Gradescope:

- `Recursive.java`
- `RecursiveTester.java`
- `LetterInventory.java`
- `AnagramSolver.java`
- `AnagramFinderTester.java`

Thank you to Professor Mike Scott for creating the previous version of this assignment!
Anagram Solver is based on an assignment created by Stuart Reges.